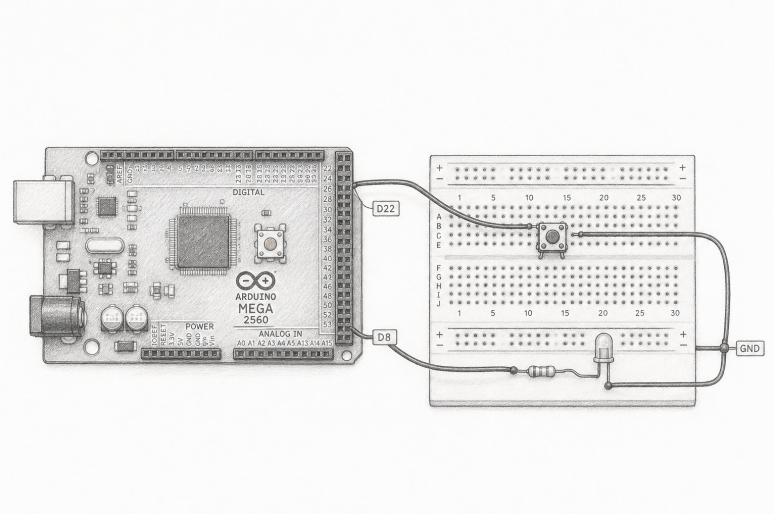
Deterministic Reaction Timer

Button + DigitalOutput on the Arduino Mega 2560

First predict, then replay, then trust the stopwatch.

Lesson	003 — integration project	**Time**	100–140 minutes
Inputs	pushbutton on D22	**Output**	LED on D8
Status	host verified; hardware experimental	**Board**	Arduino Mega 2560



Orientation only. Use the exact schematic and connection table below. Confirm the LED resistor, D8, D22, and GND before power.

1 Mission and success criteria

Build a reaction timer with a debounced button and a diagnostic LED. A press and release start the trial, a ready pulse precedes a configured dark wait, and the cue then remains on for a configured response window. A press during the dark wait is premature; no press before timeout is a failure. Given the same immutable configuration and timestamped observations, the program must reproduce the same states and result.

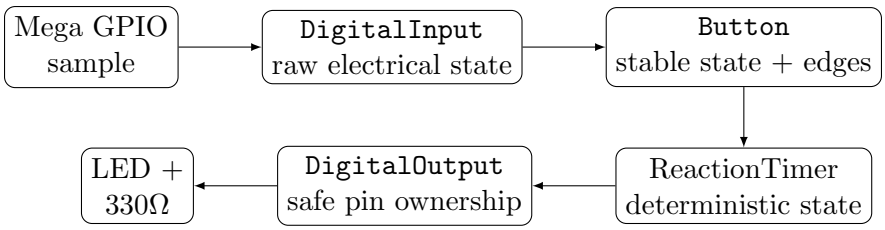
By the end, you can:

- trace an active-low button and an active-high LED from pin to ground;
- distinguish a raw electrical sample from a debounced state and edge;
- explain why time is supplied to `update()` instead of read secretly;
- replay a bounce trace and predict its single stable press event;
- test premature presses, timeout, rollover-safe elapsed time, and shutdown;
- collect a Mega 2560 acceptance record without calling observation proof of timing accuracy.

Safety checkpoint. Disconnect USB and every other power source before changing wiring. Use a 330Ω series resistor with the external LED. Do not connect an output directly to ground or 5 V. The button circuit below uses the Mega's internal pull-up: do not add an external 5 V jumper. Stop immediately for heat, odor, unexpected brightness, or repeated resets.

2 Why this is lesson 003

Lessons 001 and 002 introduced one hardware endpoint at a time. This third lesson composes them into a project:



The project *composes* owners. It does not make a button inherit from an input pin or a game inherit from an LED. Each layer gives the layer above a smaller, clearer contract.

3 Predict before wiring

The button defaults to the internal pull-up. Open means HIGH and pressed means LOW. The LED is active-high.

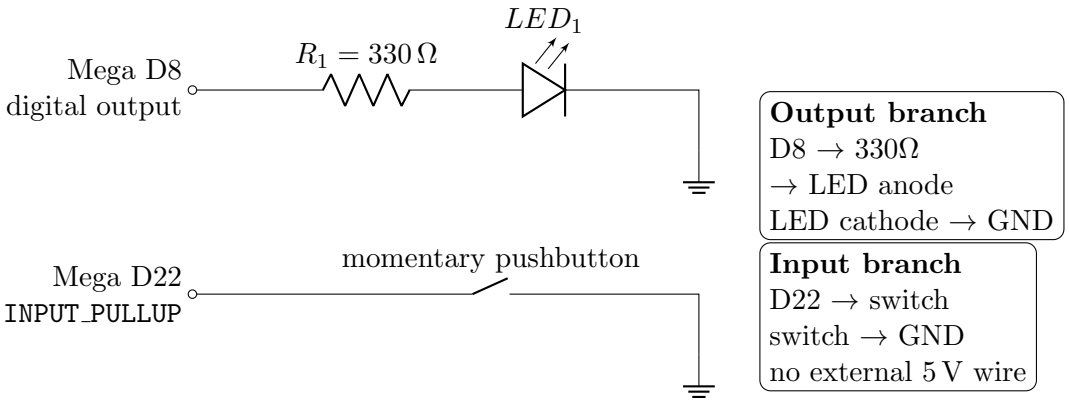
Button	D22 sample	D8 level	Meaning
released	HIGH	LOW	waiting in the dark
pressed	LOW	LOW	possible press; debounce first
released	HIGH	HIGH	cue is visible; timer is running
pressed	LOW	HIGH	possible reaction; debounce first

Predict. A dirty contact reads LOW, HIGH, LOW, LOW at 1000, 1002, 1005, and 1025 ms. With a 20 ms debounce interval, at what timestamp can a stable press first be reported? _____

Predict. Why must the cue remain off after an early press until the button has been released?

---

4 Exact schematic — build only from this page



Connection	Required part	Unpowered verification
D8 to LED anode	330Ω resistor in series	meter or color bands
LED cathode to GND	LED polarity	datasheet; flat edge is only a clue
D22 to switch terminal A	jumper	continuity to D22 only
switch terminal B to GND	jumper	open unpressed, continuous pressed

Four-pin buttons. The two pins on either side may already be connected internally. Use a continuity meter while unpowered; place the circuit across the contacts that change from open to closed when pressed.

Why D22? It is an ordinary Mega digital pin, physically separated from the common PWM cluster and serial pins. The project needs no interrupt. Polling at supplied timestamps makes behavior testable and sufficient for human input.

5 Ownership and lifetime

Construction records configuration but does not touch hardware. Initialization claims each pin and configures it. Shutdown is idempotent. The output becomes high impedance; the input releases its pull-up and claim. Destruction calls shutdown, including when a caller using exceptions unwinds through ADK code. ADK itself neither throws nor allocates.

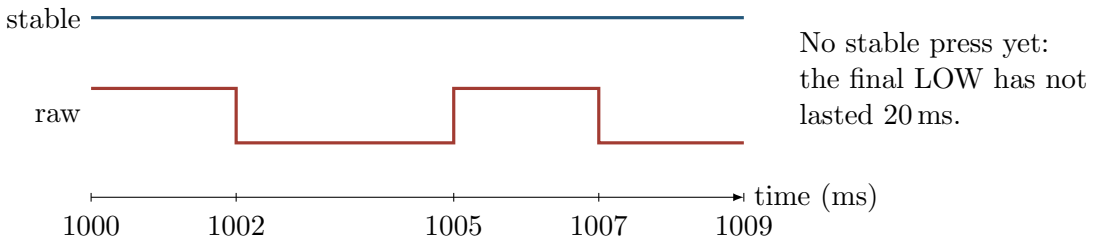
```
adk::ResourceRegistry resources;
adk::DigitalOutput    cue       (resources, 8, adk::Level::Low);

adk::Button trigger
{
    resources,
    adk::ButtonConfig
    {
        22,
        adk::Pull::Up,
        adk::Level::Low,
        adk::Duration (20)
    }
};

adk::ReactionTimerConfig timerConfig;
timerConfig.readyDuration   = adk::Duration (1000);
timerConfig.waitDuration    = adk::Duration (2000);
```

```
timerConfig.responseTimeout = adk::Duration (1500);
timerConfig.resultDuration  = adk::Duration (1000);

adk::ReactionTimer timer (timerConfig);

const bool cueReady    = cue.initialize     () == adk::Status::Ok;
const bool buttonReady = trigger.initialize () == adk::Status::Ok;
const bool timerReady  = timer.initialize   () == adk::Status::Ok;

if (!(cueReady && buttonReady && timerReady))
{
    trigger.shutdown ();
    cue.shutdown     ();
    return;
}
```

The compile-authoritative full sketch is This endpoint-focused listing uses the exact supported API. The canonical `examples/Lesson003ReactionTimer` composes the same output through `MonoLed` and obtains the registry from `Runtime`. Its `LED_BUILTIN` pin is D13; for the exact external D8 circuit in this lesson, make a local example copy and change only that constructor argument to 8. Notice the aligned calls and explicit `== Status::Ok` checks. `ReactionTimer` is a pure state owner; the GPIO owners perform hardware cleanup. No callback runs during cleanup.

6 Raw, stable, and event are different facts

A switch contact may alternate several times before settling. `rawPressed()` reports the latest interpreted candidate. `pressed()` changes only after the candidate state has remained unchanged for the configured interval. `pressEvent()` and `releaseEvent()` are non-consuming snapshots: every observer gets the same answer until the next update.



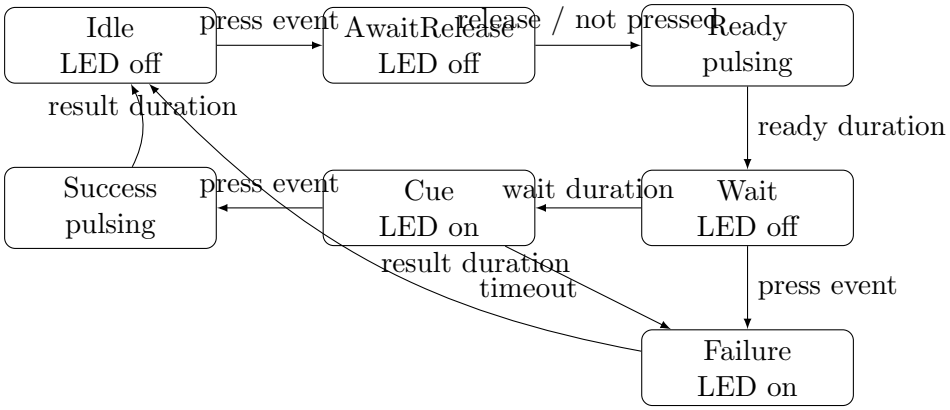
At 1029 ms, if LOW persisted, `pressed()` and `pressEvent()` are both true. Calling `pressEvent()` twice still returns true. At the next update, `pressEvent()` becomes false while `pressed()` remains true. A stable release later produces one `releaseEvent()`.

6.1 A precise debounce rule

1. Interpret the raw level using active-low or active-high configuration.
2. If raw differs from the current candidate, replace the candidate and record `candidateSince = now`.
3. If candidate differs from stable and `now - candidateSince >= debounce`, commit the stable state.
4. Derive exactly one press or release snapshot from that stable transition.

Unsigned elapsed subtraction makes the comparison safe across one clock rollover, provided every configured interval is less than half the time range. Do not compare absolute deadlines with `now >= deadline`.

7 The reaction-timer state machine



The button arms its press event only after a stable release, so a button held during initialization cannot start a trial. `AwaitRelease` also requires release before the ready phase. Only debounced events count. The cue timestamp and LED snapshot belong to the same state transition:

$$reaction = pressTimestamp - cueTimestamp.$$

Do not turn on the LED in one update and record the start in a later update. The measured value then includes an undocumented scheduling error. This first implementation uses a fixed `waitDuration`; a later randomized adapter must choose that value before construction and record its generator version and seed. It must not hide randomness inside `ReactionTimer`.

8 Reference update order

```
void loop ()
{
    const adk::TimePoint now (millis ());

    trigger.update (now);
    const adk::Status timerStatus = timer.update (now, trigger);

    const adk::ReactionTimerSnapshot state = timer.snapshot ();
    const adk::Level level = state.ledOn
                           ? adk::Level::High
                           : adk::Level::Low;
    const adk::Status outputStatus = cue.write (level);

    if (!(timerStatus == adk::Status::Ok &&
          outputStatus == adk::Status::Ok))
    {
        trigger.shutdown ();
        cue.shutdown     ();
    }
}
```

All observers see one coherent update. Input sampling precedes behavior; behavior precedes output. Reporting comes last in the canonical example. Reordering those phases can introduce a one-update disagreement between the LED and recorded start time.

9 Replay before hardware

First test `ReactionTimer` at its narrow observation boundary. Configure ready = 100 ms, wait = 1000 ms, response timeout = 1500 ms, and result = 1000 ms. Initialize it, then replay these exact observations. The button bounce trace on the previous page is tested independently and then end-to-end.

Time (ms)	button observation	state	expected snapshot
0	released	Idle	LED off; no outcome
100	pressed + press event	AwaitRelease	LED off
200	released + release event	Ready	ready pulse on
299	released	Ready	duration not yet due
300	released	Wait	LED off
1299	released	Wait	duration not yet due
1300	released	Cue	cue time = 1300; LED on
1450	pressed + press event	Success	reaction = 150 ms
2450	released	Idle	result duration elapsed

Determinism check. Run the trace twice from newly constructed objects with the same configuration. Compare every snapshot, output write, state, and result byte for byte. A matching final number alone is insufficient.

9.1 Complete these trace exercises

1. Add a press event at 800 ms. Expected state, outcome, and reason: _____.
2. Hold LOW while initializing `Button`. Identify its press-arming behavior and the first action that can start `ReactionTimer`: _____.
3. Replay times near unsigned wrap: `0xFFFFFFF0`, `0xFFFFFFFA`, `0x00000004`. Calculate each elapsed difference: _____.
4. Read `pressEvent()` twice after a stable press update. Record both: _____. Advance once and record it again: _____.

10 Minimum correctness suite

Each row needs an assertion over state, event snapshots, hardware operations, and resource claims.

Case	Required proof	Result
clean press/release	one press, one release, correct timestamps	☐
bounce shorter than 20 ms	no stable edge	☐
exactly 20 ms	edge occurs at the inclusive boundary	☐
premature press	`Failure`; outcome `PrematurePress`; cue absent	☐
held at start	trial waits for release	☐
press after cue	reaction uses supplied timestamps	☐
two event observers	identical non-consuming snapshots	☐
timestamp rollover	elapsed subtraction remains correct	☐
busy D8 or D22	initialization fails without leaked claim	☐
injected configure failure	reverse rollback; no later write	☐
repeated initialization	success without reconfiguration	☐
repeated shutdown	no unsafe duplicate operation	☐
destruction while active	D8 high impedance; claims reusable	☐
two identical replays	every observable byte matches	☐

Fault-injection exercise. Make D22 configuration fail after D8 has been claimed and configured. Write the expected cleanup operations in order:

_____.

Composition exercise. Explain why the reaction timer should receive a button snapshot rather than read D22 itself:

_____.

11 Mega 2560 procedure

1. Record repository commit, Arduino CLI version, AVR core version, and fully qualified board name `arduino:avr:mega`.
2. Disconnect power. Build the exact schematic and check every row in the connection table. Confirm the external LED resistor with a meter or bands.
3. Compile the unmodified `examples/Lesson003ReactionTimer`. Record flash and static RAM. Compilation proves only that the program fits and type-checks.
4. Copy the example to a temporary sketch and replace `LED_BUILTIN` with 8 so it matches this lesson's exact external schematic. Compile, upload, and keep the button released through reset. Verify the LED pulses during Ready, remains off during Wait, and then turns on for Cue.
5. Press once after the cue. Verify exactly one result is reported and the cue changes according to the documented terminal state.
6. Run three trials. Then deliberately press early; verify false-start indication and release gating.
7. Hold the button through reset. Verify no immediate valid reaction.
8. Invoke or reach the example's shutdown path. Measure D8: confirm it is input/high impedance rather than merely observing a dark LED.
9. Disconnect USB before any change. Inspect the board and components for abnormal temperature or damage.

Field	Record	Field	Record
Commit		Mega board ID	
CLI / AVR core		Supply / USB source	
Flash bytes		Static RAM bytes	
Meter / analyzer		D8 shutdown reading	

Trial	configured wait (ms)	cue time	press time	reaction (ms)
1				
2				
3				

Status language. If only the host tests and Mega compile pass, write "host verified; Mega compiled." Write "hardware verified" only after the physical checklist and measurements are complete.

12 Diagnose from evidence

Observation	Likely causes	First safe check
LED never lights	reversed LED, wrong row, failed init, premature press	power off; trace D8 branch
LED always lights	D8 shorted HIGH, wrong state, output not shut down	disconnect; inspect then fake trace
button always active	switch orientation, D22 grounded, polarity mismatch	power off; continuity test
one press reports many	raw state used as event, debounce reset error	replay bounce trace
reaction too small	wrong event accepted or cue timestamp recorded late	inspect button arming and transition
result varies in replay	hidden clock/random/global state	log every supplied input
board resets on cue	short or excessive current	disconnect immediately

13 Analysis and extension

1. Show the reaction calculation for each hardware trial. Report timer resolution; do not report more precision than the clock supplies.
2. Does debounce add 20 ms to the reported reaction? State whether the design records the first raw candidate or stable-transition timestamp, then defend that policy:

   _____.
3. Add a second diagnostic output without changing `Button`. Use it to distinguish Ready, Wait, Cue, Success, and Failure.
4. Replace the LED cue with a semantic `MonoLed`. Which layer should know that HIGH means luminous?

   _____.

5. Design a two-button choice reaction test. Define simultaneous stable presses as an explicit invalid chord; do not depend on update order.
6. Design a replay file containing configuration, timestamps, button observations, and expected snapshots. Then specify the generator version and seed fields a future randomized wait adapter would add.

14 Claim, evidence, reasoning

Claim. The project converts a bouncing active-low contact into exactly one deterministic reaction result while managing both pins safely.

Evidence. Cite at least one complete replay, one fault-injection test, three physical trials, and the measured D8 shutdown state.

Reasoning. Connect the evidence to the debounce interval, button arming, state transition timestamps, resource ownership, and high-impedance cleanup.

---

15 Further reading

- Arduino Mega 2560 Rev3 documentation and schematics: official Mega 2560 hardware page
- Arduino language reference, digital pins and pull-ups: `https://docs.arduino.cc/language-reference/`
- Microchip ATmega640/1280/1281/2560/2561 datasheet: `https://www.microchip.com/en-us/product/ATmega2560`
- Jack Ganssle, *A Guide to Debouncing*: `https://www.ganssle.com/debouncing.htm`
- C++ Core Guidelines, resource management: resource-management section
- ADK HTML reference and compile-authoritative examples: `https://spincyc.github.io/adk/`

Final sign-off
Wiring checked unpowered: ☐ Replay passed twice: ☐ False start tested: ☐ D8 shutdown measured: ☐ Power removed before teardown: ☐